UNICAMP - Universidade Estadual de Campinas

FEEC - Faculdade de Engenharia Elétrica e de Computação

Câmbio CVT Físico com BitDogLab V7

RELATÓRIO DE PROJETO

Link deste documento (acesso Comentador para UNICAMP): Link

Repositório no GitHub

Vídeo de demonstração

Arthur Lucas Da Silva Nogueira	RA: 213293
Raul Galdino Tancredo	RA: 223908

Prof. Eric Rohmer

Campinas
2026

1. Descrição do Sistema

O sistema desenvolvido consiste na implementação de um Câmbio de Transmissão Continuamente Variável (CVT) simulado e controlado remotamente via interface sem fio. O projeto utiliza a plataforma BitDogLab V7 como unidade central de processamento, integrando comunicação serial, controle de potência e telemetria.

Utilizando uma ponte H L293D, o controle dos motores é dividido em dois eixos de atuação:

1. Eixo motor: motor de corrente contínua (DC) responsável pela força motriz principal, conectado ao cone principal.
2. Atuador do câmbio: motor de corrente contínua acoplado a um sistema de fuso (atuador linear), responsável por alterar a posição física da correia do câmbio CVT. No sistema de fuso, há a polia (que tem o anel o-ring/elástico girando livremente) e a guia, que faz com que o movimento de rotação do fuso se transforme em movimento linear.

A interface homem-máquina (IHM) é realizada de forma remota através de um módulo Bluetooth (conectado à interface UART da placa) com a integração de um aplicativo no celular ("Arduino Bluetooth Control" na Play Store do Google). O display OLED da placa é utilizado para exibir a telemetria em tempo real.

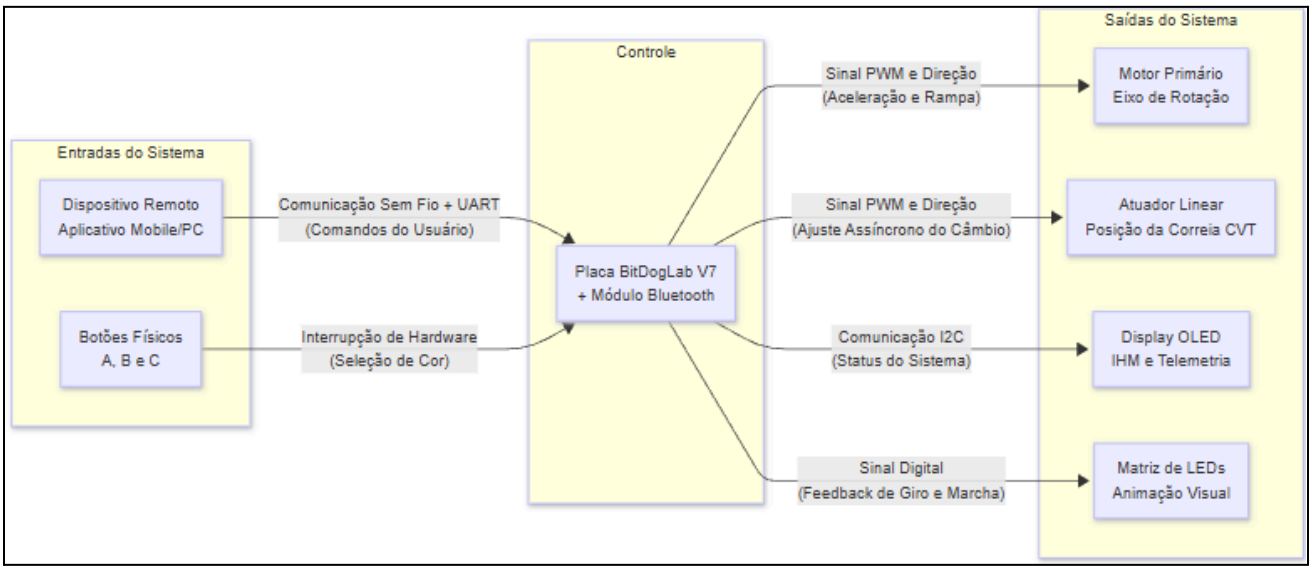

Figura 1 - Arquitetura do sistema.

2. Código fonte

- Configuração do módulo Bluetooth HC-05: /libs/hc05_atMode_config.py

```python
"""
Como usar:
1. Segurar botão da placa HC-05 (ao ligar a BitDogLab, o módulo vai piscar os LEDs
a cada 2 segundos).
2. Inserir o cabo USB na entrada da BitDogLab.
3. No VSCode, selecionar este .py na árvore usar a opção "Run current file on
Pico"
4. Testar interface com o comando "AT" --> Deve aparecer um "OK" onde estava o
"AT".
5. Configurar usuário da placa:
    a. AT+NAME=....     --> digitar o nome do módulo; e.g.: AT+NAME=EA801_CVT
    b. AT+PSWD=xxxx     --> senha de 4 digitos = xxxx
    c. AT+CMODE=1       --> para se conectar com qualquer dispositivo através da
senha
6. Desligar a placa e realizar pareamento.


A. Para mais comandos, ver /docs/datasheets/HC-05_ATCommandSet.pdf
"""



import machine
import sys
import uselect
import utime

# Configura a UART0 nos pinos 0 (TX) e 1 (RX) - Conector J2
# O baud rate de 38400 é o padrão de fábrica para o modo AT do HC-05
uart = machine.UART(0, baudrate=38400, tx=machine.Pin(0), rx=machine.Pin(1))

print("==================================")
print("     Terminal AT para HC-05       ")
print("==================================")
print("Digite 'AT' e pressione Enter.")
print("A resposta deve ser 'OK'.")
print("==================================")

# Configura a entrada do teclado (terminal USB) para não bloquear o código
teclado = uselect.poll()
teclado.register(sys.stdin, uselect.POLLIN)

# Envia um "AT" inicial de teste (igual ao seu código C++)
```

```python
uart.write('AT\r\n')

while True:
    # 1. Se chegou algo do HC-05, imprime no terminal do PC
    if uart.any():
        resposta = uart.read()
        if resposta is not None:        # sem isso vai aparecer um erro no decode
abaixo
            try:
                print(resposta.decode('utf-8'), end='')
            except UnicodeError:
                print(resposta) # Imprime cru se houver lixo na serial

    # 2. Se digitar algo no terminal do PC, 05envia para o HC-
    if teclado.poll(0):
        comando = sys.stdin.readline().strip() # Lê o que foi digitado
        if comando:
            # O HC-05 exige que os comandos AT terminem com Carriage Return e Line
Feed (\r\n)
            uart.write(comando + '\r\n')

    utime.sleep(0.01) # Pequena pausa para não sobrecarregar o processador
```

- Mapeamento de comandos Bluetooth: /libs/bluetooth_mapping.py

```python
"""
1. Arquivo de uso geral para mapear o que chega de informação na UART com o módulo
bluetooth.
2. Utilizado para reconhecer os comandos e configurar o setup.py deste projeto.
3. O que a placa receber será exibido no Terminal do VSCode.
"""

import machine
import utime

# Inicializa a UART0 para o Bluetooth (Pinos do Conector J2)
# Baud rate 9600 é o padrão para comunicação (modo transparente)
bluetooth = machine.UART(0, baudrate=9600, tx=machine.Pin(0), rx=machine.Pin(1))

# Configura o LED Azul da placa (GPIO 12) para feedback visual
# O BIH diz que é cátodo comum, então '1' liga e '0' desliga
led_azul = machine.Pin(12, machine.Pin.OUT)
led_azul.value(0) # Inicia desligado
```

```python
print("========================================")
print("   Monitor Bluetooth - Projeto CVT     ")
print("========================================")
print("Aguardando conexão e comandos do celular...")

while True:
    # Verifica se há dados no buffer do Bluetooth
    if bluetooth.any():
        # Lê os dados brutos recebidos
        dados_brutos = bluetooth.read()

        try:
            # Tenta decodificar de bytes para texto (String)
            # O .strip() remove espaços em branco e quebras de linha (\r, \n) das
pontas
            comando = dados_brutos.decode('utf-8').strip() # type: ignore

            # Só imprime se realmente tiver algo após o strip
            if comando:
                print(f"Comando recebido em Texto: '{comando}'")

                # Envia um "eco" de volta para o celular (aparece na tela do app)
                bluetooth.write(f"Placa recebeu: {comando}\n")

                # Pisca o LED Azul rapidamente
                led_azul.value(1)
                utime.sleep(0.1)
                led_azul.value(0)

        except UnicodeError:
            # Se o aplicativo do celular mandar dados hexadecimais puros
            # (comum em apps de controle tipo "joystick"), a decodificação de
texto falha.
            # Nesse caso, imprimimos os bytes puros para você poder mapear.
            print(f"Dados recebidos em Bytes (Hex): {dados_brutos}")

    # Pausa curta para não travar a CPU no loop infinito
    utime.sleep(0.05)
```

- Setup do projeto: [/libs/setup.py]

```python
"""
    arquivo de importação de bibliotecas, configuração de interfaces e variáveis
"""
```

```python
from machine import Pin, ADC, I2C, PWM
import neopixel, utime, machine
import ssd1306

# 1. Variáveis Globais e Cores
corVermelha = (20, 0, 0)
corAzul = (0, 0, 20)
corVerde = (0, 20, 0)
cor_travada = corVermelha  # Motor começa vermelho
ultimo_tempo_btn = 0
ultimo_tempo_pisca = utime.ticks_ms()
indice_frame = 0

# 2. Variáveis de Controle do Motor Virtual
CENTRO_JOYSTICK = 32768
ZONA_MORTA = 4000
MAX_LEDS_POR_SEGUNDO = 100  # Ajuste a velocidade máxima aqui

# 3. Configuração dos Botões (A, B e C)
botao_a = Pin(5, Pin.IN, Pin.PULL_UP)
botao_b = Pin(6, Pin.IN, Pin.PULL_UP)
botao_c = Pin(10, Pin.IN, Pin.PULL_UP)

# 4. Configuração do Joystick
joystick_x = ADC(Pin(27))
joystick_y = ADC(Pin(26))
joystick_sw = Pin(22, Pin.IN, Pin.PULL_UP)

# 5. Configuração do Display OLED (SSD1306)
# Usando os pinos I2C0 da BitDogLab V7 (SDA=2, SCL=3)
i2c = I2C(1, scl=Pin(3), sda=Pin(2), freq=400000)
display = ssd1306.SSD1306_I2C(128, 64, i2c, addr=0x3C)
display.fill(0)
display.show()
ATT_DISPLAY_MS = 100

# 6. Configuração da Matriz de LEDs
NUM_LEDS = 25
np = neopixel.NeoPixel(Pin(7), NUM_LEDS)
LED_MEIO = 12
# Buffer circular da animação do motor
FRAMES_MOTOR = [[14, 13], [15, 16], [16, 24], [16, 23], [17, 22],
                [18, 21], [18, 20], [18, 19], [10, 11], [8, 9],
```

```
                [0, 8], [1, 8], [2, 7], [3, 6], [4, 6], [5, 6] ]

# 7. Configuração dos controles de Motor (Ponte H) e PWM da placa
# IMPORTANTE: GPIO 0 e 1 são do Bluetooth. Vamos usar os pinos livres do conector
IDC.
# Motor Principal (Eixo)
Mprincipal_DIR1 = Pin(16, Pin.OUT)
Mprincipal_DIR2 = Pin(17, Pin.OUT)
Mprincipal_PWM  = PWM(Pin(18))
Mprincipal_PWM.freq(1000)
Mprincipal_PWM.duty_u16(0)
TEMPO_RAMPA_MS = 1000
# Motor Câmbio (Atuador CVT)
Mcambio_DIR1 = Pin(19, Pin.OUT)
Mcambio_DIR2 = Pin(8, Pin.OUT)
Mcambio_PWM  = PWM(Pin(9))
Mcambio_PWM.freq(1000)
Mcambio_PWM.duty_u16(0)
TEMPO_PASSO_CVT_MS = 2000  # 2000 ms = 2 segundos

# 8. Configura o LED Azul da placa (GPIO 12) para feedback visual
led_azul = machine.Pin(12, machine.Pin.OUT)
led_azul.value(0) # Inicia desligado

# 9. Inicializa a UART0 para o Bluetooth (Pinos do Conector J2)
bluetooth = machine.UART(0, baudrate=9600, tx=machine.Pin(0), rx=machine.Pin(1))
status_bt = machine.Pin(4, machine.Pin.IN)      # Pino que vai ler o pino 'STATE'
do módulo Bluetooth

# 10. Mapeamento das entradas bluetooth
outButtonDir = '0'
frontDir = 'F'
backDir = 'B'
rightDir = 'R'
leftDir = 'L'
triangleDir = 'T'
circleDir = 'C'
squareDir = 'S'
crossDir = 'X'
startDir = 'A'
pauseDir = 'P'
```

- Arquivo principal: /src/main.py

```python
from setup import *

# ==========================================
# FUNÇÕES DE HARDWARE (MOTORES E TELA)
# ==========================================

# Lida com as informações do display OLED
def printOled(velocidade, direcao, posicao_cvt, bt_conectado):
    texto_sentido = "Horario" if direcao == 1 else "Anti-Hor."
    texto_sentido = "Parado" if velocidade == 0 else texto_sentido
    texto_bt = "ON" if bt_conectado == 1 else "OFF"
    display.fill(0)
    display.text("Cambio CVT", 20, 0)
    display.text("-" * 16, 0, 10)
    display.text(f"Motor: {texto_sentido}", 0, 25)
    display.text(f"Pos. CVT: {posicao_cvt}%", 0, 40)
    display.text(f"Bluetooth: {texto_bt}", 0, 55)
    display.show()

#controle do motor principal, lidando com a velocidade (mantida em 100%) e sentido
de giro
def controlar_motor_principal(velocidade_percentual, direcao, fazer_rampa=False):
    # verificação se recebeu o comando para inverter o sentido do motor ou parar
    # nos dois casos tem rampa de variação de velocidade para não ter um solavanco
nos componentes mecânicos
    if fazer_rampa and velocidade_percentual > 0:
        passos = 10
        tempo_por_passo = (TEMPO_RAMPA_MS / 2000) / passos
        passo_pwm = max(1, velocidade_percentual // passos)
        for v in range(velocidade_percentual, -1, -passo_pwm):
            Mprincipal_PWM.duty_u16(int((v / 100) * 65535))
            utime.sleep(tempo_por_passo)
        Mprincipal_DIR1.value(0)
        Mprincipal_DIR2.value(0)
        Mprincipal_PWM.duty_u16(0)
        utime.sleep(0.05)
        if direcao == 1:                # a mudança de direção pode ser feita na
inversão dos fios de alimentação do motor
            Mprincipal_DIR1.value(1)
            Mprincipal_DIR2.value(0)
        else:
            Mprincipal_DIR1.value(0)
            Mprincipal_DIR2.value(1)
        for v in range(0, velocidade_percentual + 1, passo_pwm):
```

```
            Mprincipal_PWM.duty_u16(int((v / 100) * 65535))
            utime.sleep(tempo_por_passo)
        Mprincipal_PWM.duty_u16(int((velocidade_percentual / 100) * 65535))
    else:           # controle normal, apenas ocorre se recebe algum comando via
Bluetooth
        if velocidade_percentual > 0:
            if direcao == 1:
                Mprincipal_DIR1.value(1)
                Mprincipal_DIR2.value(0)
            else:
                Mprincipal_DIR1.value(0)
                Mprincipal_DIR2.value(1)
        else:
            Mprincipal_DIR1.value(0)
            Mprincipal_DIR2.value(0)
        Mprincipal_PWM.duty_u16(int((velocidade_percentual / 100) * 65535))

# movimentação do câmbio, dependendo do sentido que ele precisar girar
# velocidade também sempre em 100%
def iniciar_movimento_cvt(direcao):
    if direcao == 1:
        Mcambio_DIR1.value(1)
        Mcambio_DIR2.value(0)
    elif direcao == -1:
        Mcambio_DIR1.value(0)
        Mcambio_DIR2.value(1)
    else:
        return
    Mcambio_PWM.duty_u16(65535)

# ==========================================
# Função de Interrupção dos Botões
# ==========================================
def trata_interrupcao_botao(pino):
    global cor_travada, ultimo_tempo_btn
    tempo_atual = utime.ticks_ms()
    if utime.ticks_diff(tempo_atual, ultimo_tempo_btn) > 200:
        if pino == botao_a:
            cor_travada = corVermelha
        elif pino == botao_b:
            cor_travada = corAzul
        elif pino == botao_c:
            cor_travada = corVerde
        ultimo_tempo_btn = tempo_atual
```

```python
botao_a.irq(trigger=Pin.IRQ_FALLING, handler=trata_interrupcao_botao)
botao_b.irq(trigger=Pin.IRQ_FALLING, handler=trata_interrupcao_botao)
botao_c.irq(trigger=Pin.IRQ_FALLING, handler=trata_interrupcao_botao)

# ==========================================
# INICIALIZAÇÃO E LOOP PRINCIPAL
# ==========================================
ultimo_tempo_pisca = utime.ticks_ms()
ultimo_tempo_oled = utime.ticks_ms()

# inicializacao dos motores
velocidade_atual = 0
direcao_motor = 1
posicao_cvt = 0

# VARIÁVEIS PARA O CVT ASSÍNCRONO
cvt_em_movimento = False
ultimo_tempo_cvt = 0

# desliga a matriz de LEDs
for i in range(NUM_LEDS):
    np[i] = (0, 0, 0)
np.write()

print("Aguardando comandos Bluetooth...")

while True:
    tempo_atual = utime.ticks_ms()

    # 1. ATUALIZAÇÃO DO DISPLAY OLED
    if utime.ticks_diff(tempo_atual, ultimo_tempo_oled) >= ATT_DISPLAY_MS:
        estado_conexao = status_bt.value()
        if estado_conexao == 0 and velocidade_atual > 0:
            passos = 10
            tempo_por_passo = (TEMPO_RAMPA_MS / 2000) / passos
            passo_pwm = max(1, velocidade_atual // passos)
            for v in range(velocidade_atual, -1, -passo_pwm):
                Mprincipal_PWM.duty_u16(int((v / 100) * 65535))
                utime.sleep(tempo_por_passo)
            velocidade_atual = 0
            controlar_motor_principal(0, direcao_motor)
        printOled(velocidade_atual, direcao_motor, posicao_cvt, estado_conexao)
        ultimo_tempo_oled = tempo_atual

    # 2. LEITURA DE COMANDOS BLUETOOTH
```

```
    if bluetooth.any():
        try:
            dados_brutos = bluetooth.read()
            if dados_brutos is not None:
                comando = dados_brutos.decode('utf-8').strip().upper()

                if comando:
                    led_azul.value(1)

                    if comando == startDir:         # ligar motor principal
                        velocidade_atual = 100
                        controlar_motor_principal(velocidade_atual, direcao_motor)

                    elif comando == frontDir:       # sentido horário
                        deve_fazer_rampa = (direcao_motor == -1 and
velocidade_atual > 0)
                        direcao_motor = 1
                        controlar_motor_principal(velocidade_atual, direcao_motor,
deve_fazer_rampa)

                    elif comando == backDir:        # sentido anti-horário
                        deve_fazer_rampa = (direcao_motor == 1 and
velocidade_atual > 0)
                        direcao_motor = -1
                        controlar_motor_principal(velocidade_atual, direcao_motor,
deve_fazer_rampa)

                    elif comando == triangleDir:    # sobe marcha; motor CVT
                        # Adicionada a regra: "not cvt_em_movimento" para evitar
acúmulo de comandos
                        if posicao_cvt < 100 and not cvt_em_movimento:
                            iniciar_movimento_cvt(1)
                            cvt_em_movimento = True
                            ultimo_tempo_cvt = tempo_atual # Marca a hora que
ligou
                            posicao_cvt = min(100, posicao_cvt + 10)

                    elif comando == crossDir:       # desce marcha; motor CVT
                        if posicao_cvt > 0 and not cvt_em_movimento:
                            iniciar_movimento_cvt(-1)
                            cvt_em_movimento = True
                            ultimo_tempo_cvt = tempo_atual # Marca a hora que
ligou
                            posicao_cvt = max(0, posicao_cvt - 10)
```

```
                    elif comando == pauseDir:       # para o motor principal
                        velocidade_atual = 0
                        controlar_motor_principal(0, direcao_motor)
                        # Parada de emergência para o Câmbio também!
                        Mcambio_DIR1.value(0)
                        Mcambio_DIR2.value(0)
                        Mcambio_PWM.duty_u16(0)
                        cvt_em_movimento = False

                    led_azul.value(0)
        except UnicodeError:
            pass

    # 3. VERIFICADOR ASSÍNCRONO DO MOTOR CVT
    # Fica verificando se o tempo do motor do câmbio já deu
    if cvt_em_movimento:
        if utime.ticks_diff(tempo_atual, ultimo_tempo_cvt) >= TEMPO_PASSO_CVT_MS:
            # O tempo do passo (ex: 2s) acabou! Desliga o motor do CVT.
            Mcambio_DIR1.value(0)
            Mcambio_DIR2.value(0)
            Mcambio_PWM.duty_u16(0)
            cvt_em_movimento = False

    # 4. ANIMAÇÃO DA MATRIZ DE LEDS PARA REPLICAR O SENTIDO DE GIRO DO MOTOR
PRINCIPAL
    np.fill((0, 0, 0))
    if posicao_cvt == 0 or velocidade_atual == 0:
        np[LED_MEIO] = cor_travada
    else:
        leds_por_segundo = (posicao_cvt / 100) * MAX_LEDS_POR_SEGUNDO
        leds_por_segundo = .1 if leds_por_segundo < 0.1 else leds_por_segundo
        intervalo_ms = int(1000 / leds_por_segundo)

        if utime.ticks_diff(tempo_atual, ultimo_tempo_pisca) >= intervalo_ms:
            if direcao_motor == 1:
                indice_frame = (indice_frame + 1) % len(FRAMES_MOTOR)
            else:
                indice_frame = (indice_frame - 1) % len(FRAMES_MOTOR)

            ultimo_tempo_pisca = tempo_atual

        for led in FRAMES_MOTOR[indice_frame]:
            np[led] = cor_travada
    np.write()
```

```
    utime.sleep(0.01)
```

Para que a pasta /libs/ seja reconhecida ao clonar o repositório, é necessário alterar/criar o arquivo settings.json na pasta .vscode /.vscode/settings.json com a linha "./libs":

```JSON
{
    "python.languageServer": "Pylance",
    "python.analysis.typeCheckingMode": "basic",
    "python.analysis.diagnosticSeverityOverrides": {
        "reportMissingModuleSource": "none"
    },
    "python.terminal.activateEnvironment": false,
    "micropico.openOnStart": true,
    "python.analysis.typeshedPaths": [
        "~/.micropico-stubs/included"
    ],
    "python.analysis.extraPaths": [
        "~/.micropico-stubs/included",
        "./libs"
    ]
}
```

3. Projeto CAD 3D

Com o auxílio de um software de modelagem 3D, foram criadas as peças para impressão 3D dos componentes do projeto: base, cones, polias (de saída e da guia do câmbio), guia e acoplamento do câmbio.

Com os arquivos prontos, é necessário gerar para cada uma das peças um arquivo .stl que é utilizado nas impressoras 3D. Para este projeto, os arquivos podem ser encontrados em:

- Arquivos .stl e PDF 3D: /docs/CAD_files.

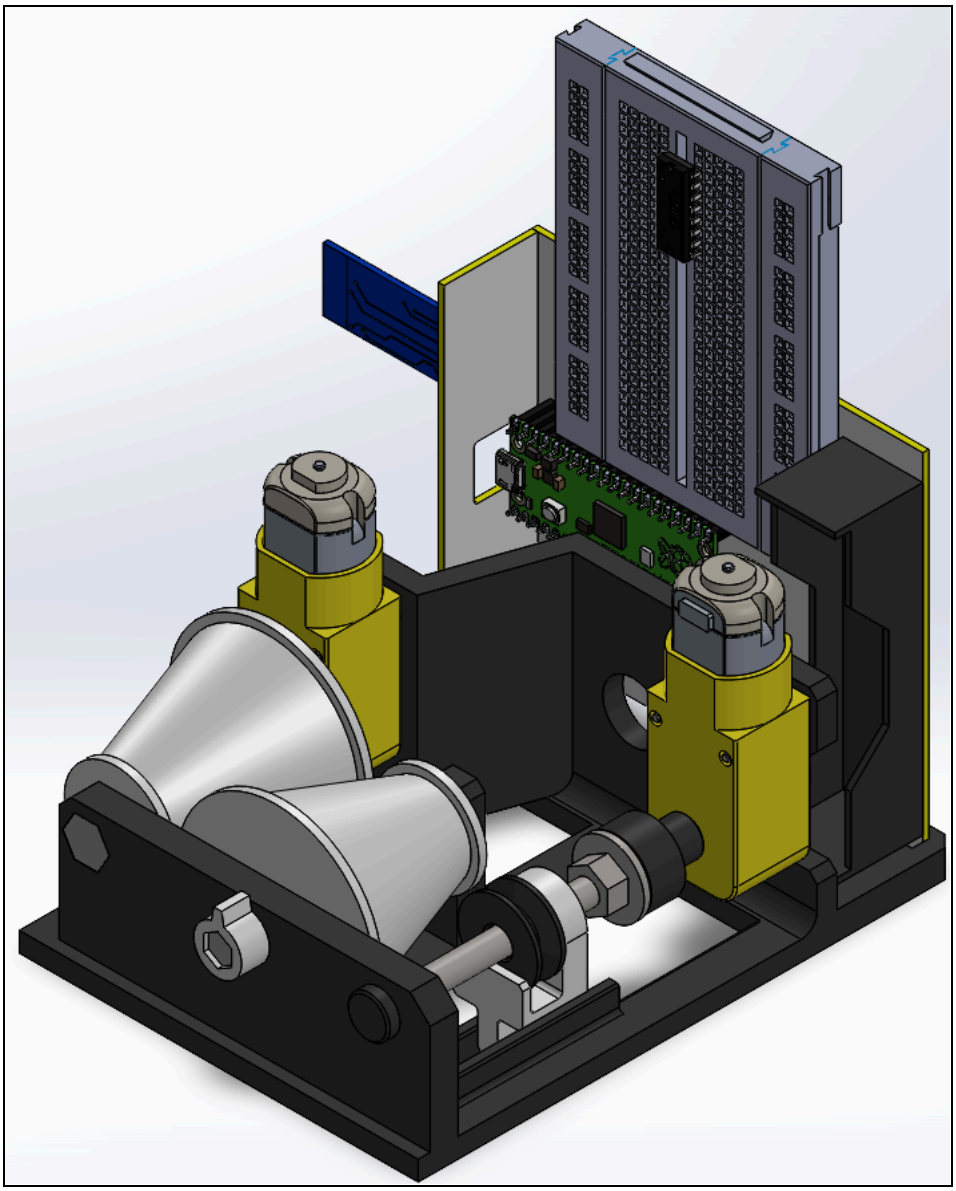

Figura 2 - Montagem 3D do projeto preliminar.

4. Montagem e desvios de projeto

Previamente à manufatura das peças em 3D, realizaram-se os testes de bancada para validar a programação e o acionamento dos motores. Após a validação de ambos os atuadores, o sistema de controle com a ponte H foi consolidado em uma placa de fenolite. O objetivo dessa montagem foi conferir estabilidade física aos componentes elétricos e mitigar o risco de rompimento dos cabos quando o sistema fosse submetido a vibrações.

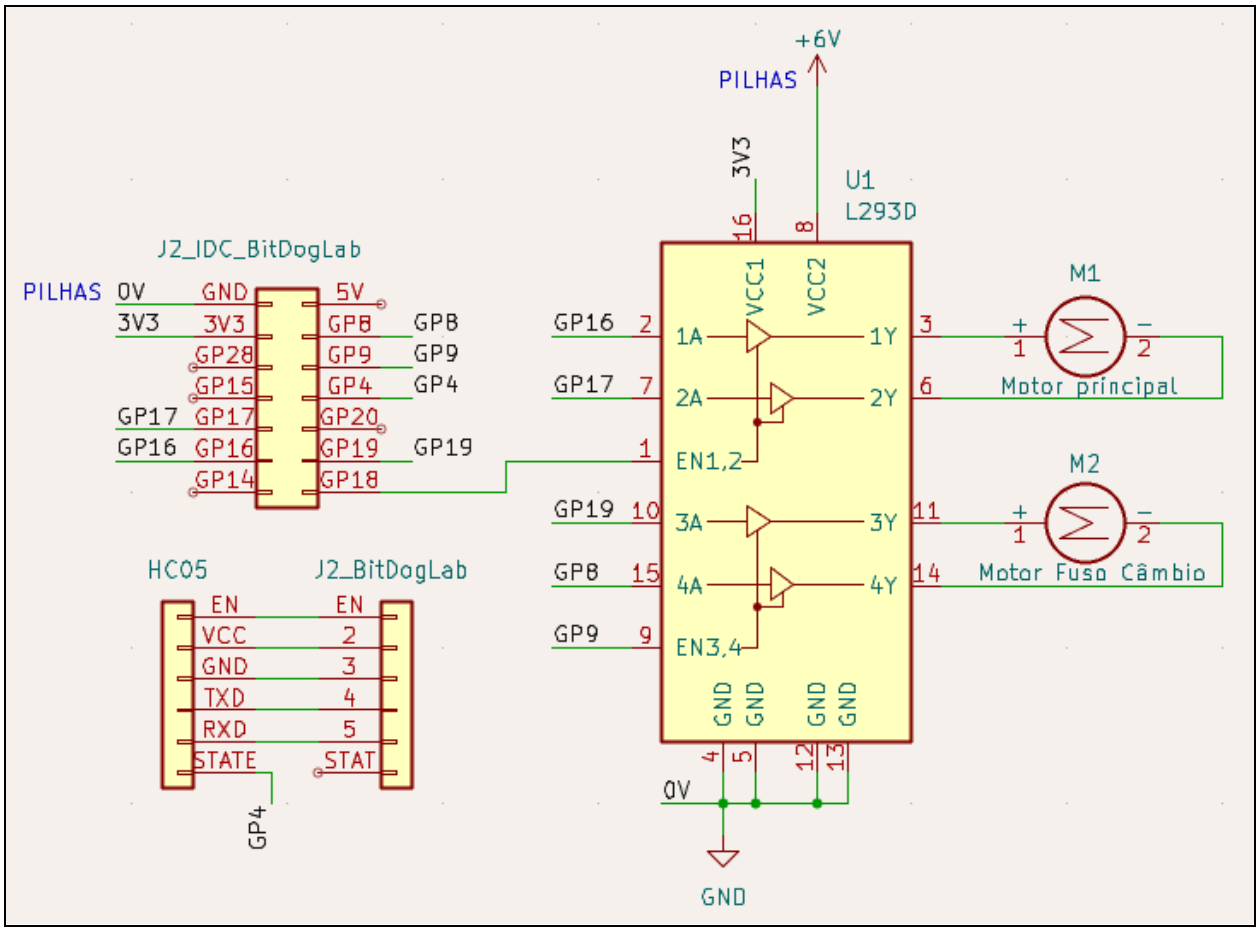

Figura 2 - Esquemático da placa auxiliar para o conector IDC.

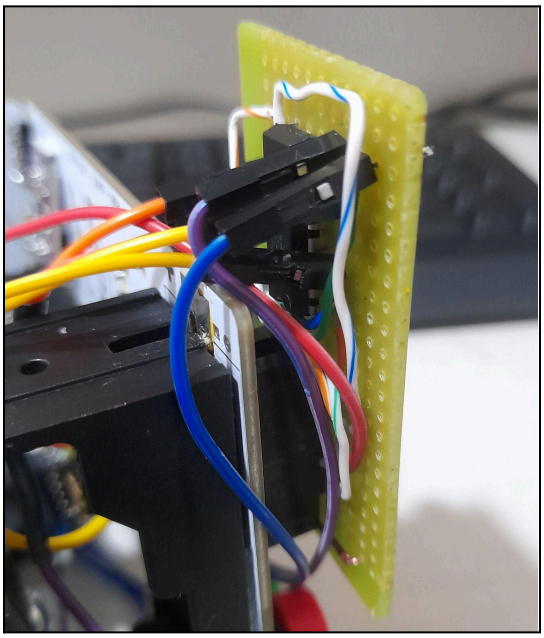

Figura 3 - Placa com a ponte H e os conectores para encaixar no conector IDC.

Foi adaptado um carregador de pilhas AA/AAA para que ele fosse utilizado como adaptador de 4 pilhas AA para 6 Vcc para alimentação dos motores. Foi feita uma tentativa de alimentação da placa com estes 6V com o auxílio de 2 diodos 1N4007 (gerando 4,6V para o pino 5V do conector IDC), mas as pilhas se provaram insuficientes para a carga da placa e dos dois motores.

Figura 4 - Adaptador de pilhas AA para 6 Vcc.

Para a conexão do módulo Bluetooth, a BitDogLab conta com o jumper J2. A única alteração que precisou ser feita é em relação ao pino de estado da conexão bluetooth. Para monitorar se o sistema estava conectado ao celular ou não, foi necessário remover apenas o pino STATE do jumper J2 e colocá-lo na GP4 pois a BitDogLab não possui nenhuma conexão deste pino com alguma entrada da placa.

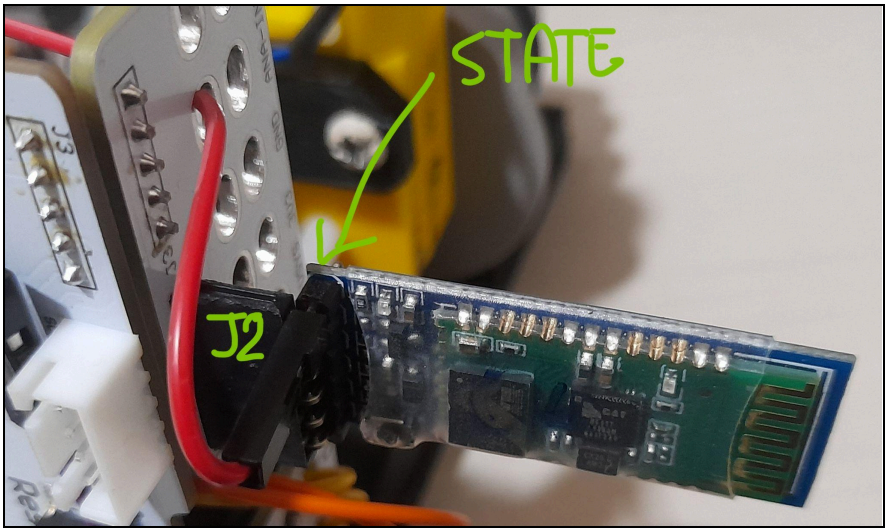

Figura 5 - Adaptação no módulo HC-05 e jumper J2.

Após a impressão das peças, notou-se que um dos arquivos .stl não foi enviado: a guia da polia do câmbio. Esta peça é responsável por fazer com que uma porca M6 no fuso não gire junto com sua rotação. Assim, o movimento circular realizado pelo fuso é transformado em movimento linear. Para mitigar este problema sem atrasar os testes, foi feito um suporte com prendedores de roupa.

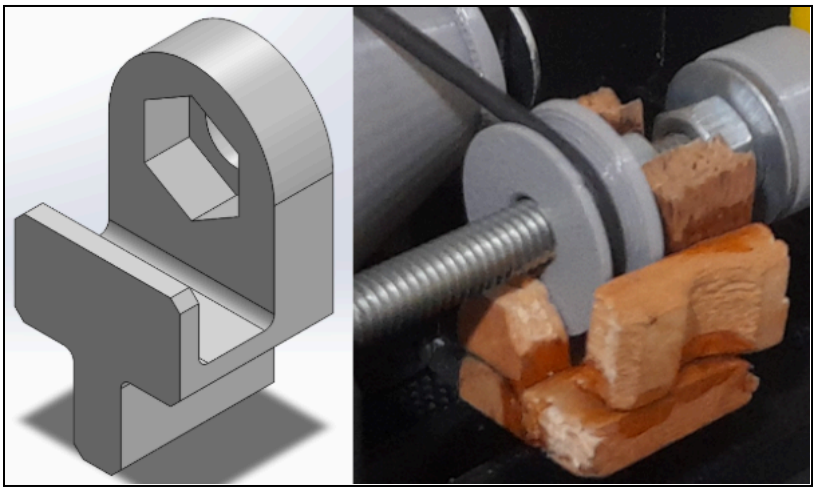

Figura 6 - Comparativo entre a peça projetada e a peça fabricada para os testes.

5. Testes realizados

Os motores foram testados individualmente antes das peças 3D ficarem prontas, validando a lógica de controle do equipamento. O mesmo é válido para a placa que substituiu a protoboard.

Com a confecção das peças em 3D e montagem do sistema, constatou-se que os cones estavam raspando na base e nos parafusos dos eixos. Com as ferramentas adequadas, a peça base foi retrabalhada e os furos centrais dos cones foram aumentados. Com estas alterações, o projeto 3D foi *asbuiltado* para que numa futura montagem não ocorram os mesmos problemas.

Resolvidos os problemas de interferência das peças, foram feitos alguns testes com o controle dos motores e foi encontrado apenas um problema crítico ao projeto: a guia do câmbio.

Como a guia está em uma extremidade do sistema, o anel o-ring utilizado como "*correia*" não fica perfeitamente reto ao longo da sua passagem pelos cones. Assim, quanto mais próximo da posição 0%, maior a divergência no tracionamento do cone de saída.

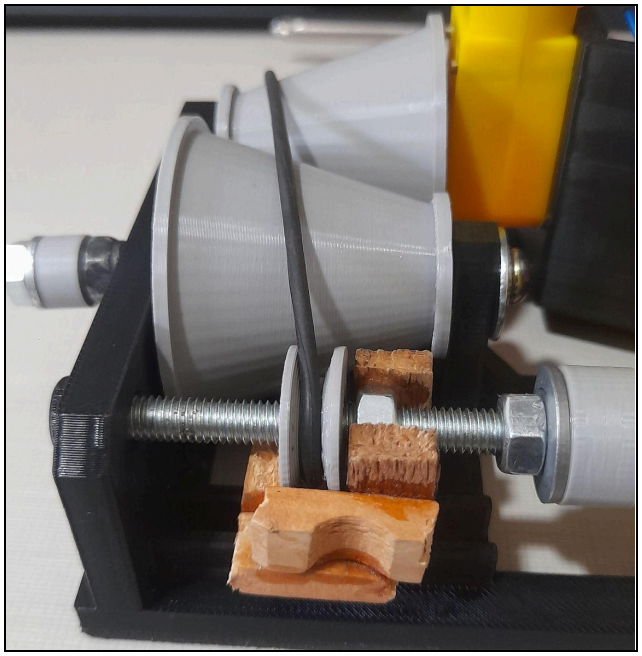

Figura 7 - Anel o-ring com desalinhamento nos cones.

Uma solução para este problema seria ter um segundo fuso no outro lado do motor principal, onde existiria outra polia e guia de câmbio para agir de maneira sincronizada com o motor do câmbio já existente.

Para testar o sistema, foi utilizado um aplicativo da Play Store do Google para se conectar ao módulo HC-05. Com o mapeamento correto realizado em [setup.py](), o controle do sistema fica conforme a imagem abaixo:

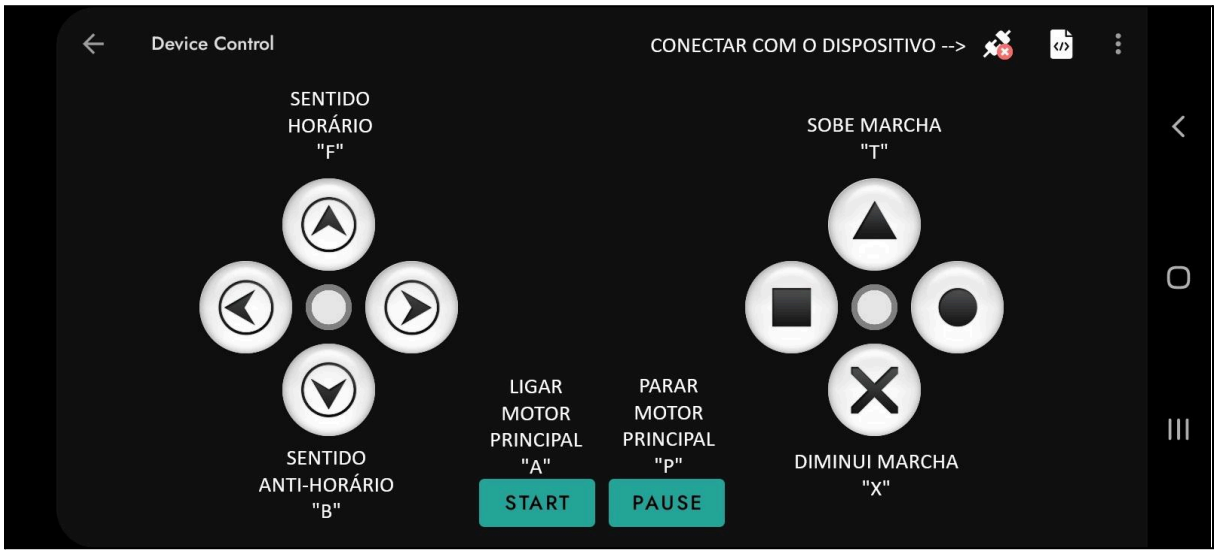

Figura 8 - Aplicativo para controle Bluetooth do câmbio CVT.

Após alguns testes, foi constatado a transferência de movimento de maneira *satisfatória* quando a polia guia do o-ring fica na região central dos cones.

Figura 9 - Sistema completo.

6. Conclusão

O projeto desmontou a viabilidade de desenvolver e controlar um protótipo funcional de um câmbio continuamente variável (CVT) utilizando a BitDogLab e sua interface UART para controle, com o uso de um módulo bluetooth HC-05.

A implementação do controle em malha aberta com operações assíncronas garantiu uma resposta rápida aos comandos remotos via Bluetooth, enquanto o uso de sinais PWM com rampas de aceleração/desaceleração provou-se fundamental para preservar a integridade mecânica dos motores e do sistema de fuso. Além disso, com os feedbacks visuais através do display OLED e matriz de LEDs, o sistema conta com uma IHM (interface homem-máquina) com telemetria clara e em tempo real.

No aspecto mecânico, a conversão do movimento rotacional em linear por meio do sistema de fuso e porca atuou de forma satisfatória para a troca das relações de marcha. Porém, na fase de testes práticos foram encontradas limitações estruturais do protótipo, destacando-se o desalinhamento do anel o-ring (*correia*).

Em resumo, o projeto atingiu seu objetivo educacional e técnico, consolidando conceitos de sistemas embarcados, cinemática, eletrônica de potência e programação estruturada em um dispositivo mecatrônico de aplicação real.

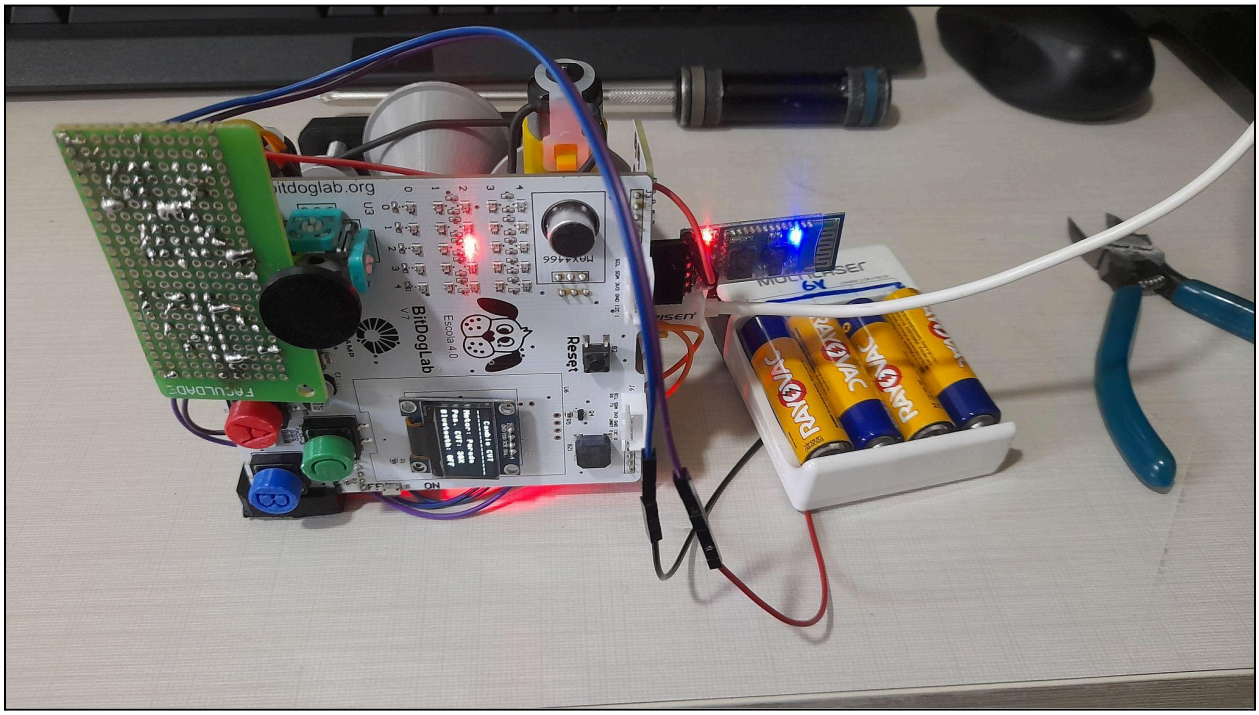

Figura 10 - Sistema funcionando.

7. Referências

- **FRUETT, F.** *BitDogLab V7*: repositório oficial. Campinas: Unicamp, 2024. Disponível em: https://gitlab.unicamp.br/fabiano/bitdoglab-v7.
- **FRUETT, F.** *Banco de informações da BitDogLab V7*. [S. l.], 2024. Disponível em: https://docs.google.com/document/d/13-68OqiU7ISE8U2KPRUXT2ISeBI3WPhXjGDFH52eWIU.
- **r223908; a213293.** *PJ2 EA801*: repositório de desenvolvimento do projeto. GitHub, 2026. Disponível em:https://github.com/r223908/pj2_ea801_r223908_a213293.